

# AquaShop — Architecture

A polyglot Kubernetes platform for a freshwater aquarium shop.  
Designed for AWS · validated with `terraform test` · run on k3d.

## READ THIS BEFORE ANYTHING ELSE

The AWS infrastructure layer in this repository has **never been applied to an AWS account**. It is written as production-quality Terraform and validated in CI with `fmt`, `validate`, `tflint`, `checkov` and `terraform test` against mock providers. That proves module composition, variable contracts and policy compliance. It does not prove AWS behaviour.

The runtime is a single-node **k3d** cluster inside WSL2 on a 16 GB laptop. Every diagram in this document carries AWS labels because AWS is the target platform, and every place where the local setup diverges from that target is named in the local-to-cloud section rather than glossed over.

The short version, said out loud without hedging: **"I built it AWS-first and ran it locally because I was paying for it myself."**

PHASE 1 COMPLETE

PHASES 2-7 PLANNED

AWS: VALIDATED, NOT APPLIED

# I The product, and why this domain

AquaShop sells live freshwater fish, shrimp, plants, hardscape, equipment and food, with a full care profile published for every living species before purchase. The domain was chosen because live livestock imposes constraints a generic shop demo does not have, and each constraint forces an architectural decision that is worth defending in an interview.

## DOMAIN CONSTRAINT → ARCHITECTURAL CONSEQUENCE

|                         |                                                                                                                                                                                                                                         |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Perishable stock</b> | Livestock ships only inside a safe weather window, so an order can be <i>confirmed but not yet shippable</i> . The order state machine has a waiting state that a normal shop does not, and it is time-driven rather than event-driven. |
| <b>Stock per tank</b>   | Stock is held per physical tank, not as one integer. Reservations are tank-scoped, time-bounded and idempotent, which is a distributed-systems problem (TTL expiry, retry safety) rather than a decrement.                              |
| <b>Compatibility</b>    | Species are not mutually compatible. That is a rules engine over water parameters and temperament, not CRUD — which is what justifies a separate service with a different language and a different change cadence.                      |
| <b>Water parameters</b> | pH, dGH and temperature ranges must <i>overlap</i> across every inhabitant. Interval intersection over a set, evaluated against tank volume.                                                                                            |
| <b>DOA claims</b>       | High-value livestock sometimes arrives dead, so there is a claims workflow with photo evidence and a compensating refund path — a real saga compensation, not a contrived one.                                                          |

## EIGHT SERVICES, AND THE PROPERTY THAT JUSTIFIES EACH LANGUAGE

| SERVICE                     | STACK                 | WHY THIS STACK                                                                                                                                                                                        |
|-----------------------------|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>catalog-service</b>      | Java 21 / Spring Boot | Read-heavy relational data with a rich schema; the ecosystem I know best, so the platform work is the novel part, not the language.                                                                   |
| <b>order-service</b>        | Java 21 / Spring Boot | Long-lived transactional orchestration; mature saga and scheduling libraries.                                                                                                                         |
| <b>inventory-service</b>    | Go                    | Many small concurrent hold/expire operations; goroutines and a 40 MB footprint suit a service that scales on request count, not CPU work.                                                             |
| <b>payment-service</b>      | Rust / Axum           | Smallest surface, strictest correctness. Exhaustive compile-time matching over ledger state transitions and non-wrapping integer arithmetic. <b>Cost: slowest CI build, smallest maintainer pool.</b> |
| <b>aquatics-advisor</b>     | Python / FastAPI      | Rules and numeric range logic that changes often and is edited by domain people, not platform people.                                                                                                 |
| <b>notification-service</b> | Go                    | I/O-bound fan-out to email and webhooks with per-target retry.                                                                                                                                        |
| <b>staff-portal</b>         | JSP / WildFly         | Deliberate. Containerising a slow-starting WAR — startup probes, stdout logging, ConfigMap config, graceful shutdown — is the exercise that connects this project to production work I actually do.   |
| <b>storefront</b>           | TypeScript / Fastify  | A BFF is I/O aggregation; server-rendered HTML avoids a client state layer that would add build complexity and prove nothing.                                                                         |

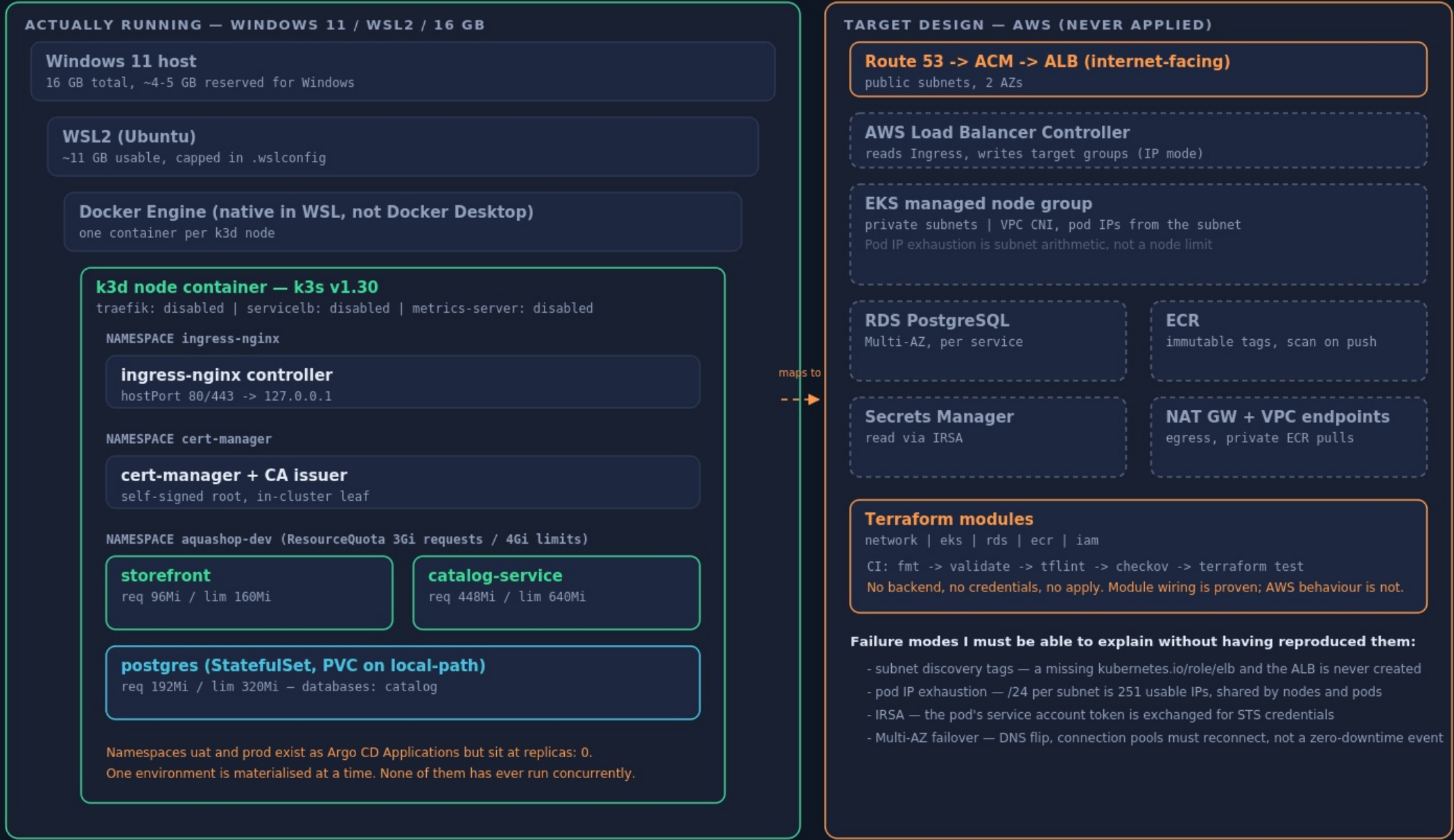
## System architecture

# AquaShop System Architecture

Target design. Phase 1 components are built; the rest are planned.

# Deployment Topology

What actually runs, beside what it is designed to become.



Total measured footprint: fill in from `kubectl top pods -A` after the first bootstrap. Estimate for this profile: ~2.6 GB.

AquaShop / docs/diagrams/deployment.svg





# | The memory budget — the constraint that shaped the design

16 GB of RAM, roughly 11 GB usable inside WSL2 after leaving Windows its share. The full platform does not fit at once, so the cluster is built as **profiles**: named subsets brought up for a purpose. This is not a workaround bolted on at the end; it is why NATS replaced Kafka, why one Postgres instance hosts per-service databases, and why only one environment is materialised at a time.

### PROFILES

| PROFILE       | ADDS                                                             | EST. TOTAL |
|---------------|------------------------------------------------------------------|------------|
| core          | k3d, ingress-nginx, cert-manager, Postgres, catalog, storefront  | ~2.6 GB    |
| commerce      | order, inventory, payment, advisor, NATS                         | ~4.2 GB    |
| full-app      | notification, staff-portal (WildFly)                             | ~5.2 GB    |
| platform      | Argo CD                                                          | ~6.1 GB    |
| observability | kube-prometheus-stack, OTel collector, Tempo, prometheus-adapter | ~9.2 GB    |

Rule enforced by the bootstrap script: never build images while the observability profile is up. ~1.8 GB of headroom does not survive a Maven or Cargo build, and the failure mode is the kernel OOM killer taking an unrelated pod.

### PHASE 1 MEASURED FOOTPRINT

Every number above is an estimate until measured. After the first bootstrap.sh run, record `kubectl top pods -A` here and in RELEASE-NOTES.md. Placeholders are left deliberately: a measured number I did not measure is the kind of claim that collapses under one follow-up question.

### DECISIONS THIS CONSTRAINT FORCED, AND WHAT EACH COSTS

|                                               |                                                                                                                                                                                                                                                                                                                                   |
|-----------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>NATS JetStream</b> instead of Kafka        | No consumer in this system needs log replay or partition ordering, and Kafka costs ~1 GB this machine does not have. Cost: no partition-key ordering, a smaller tooling ecosystem, and MSK is a heavier migration than the mapping suggests.                                                                                      |
| <b>One Postgres,</b> database per service     | Separate databases and login roles, each with CONNECT on its own database only. Saves ~1.1 GB. Cost: blast-radius isolation is lost — one restart takes every service down, which per-service RDS instances would not. Credential isolation and the no-shared-schema rule are unaffected.                                         |
| <b>One environment</b> materialised at a time | dev, uat and prod are real Argo CD Applications; two of them sit at replicas: 0. Cost: no environment has ever run concurrently with another, so cross-environment concerns are untested. Say "three overlays under GitOps, one materialised", never "I ran prod".                                                                |
| <b>No CPU limits</b> on JVM services          | A CPU limit means CFS throttling: once the container exhausts its quota it is stopped for the remainder of each 100 ms period, which appears as latency spikes while the node looks idle. Memory is limited because memory is not compressible. Cost: a runaway pod can starve its neighbours; the ResourceQuota is the backstop. |

# | Local to cloud — Phase 1 components

The full mapping document covers every component, the Terraform that would provision it, and the traffic path end to end. This page is the Phase 1 extract: what is faithful, and what I have therefore *not* validated.

| LOCAL                                 | AWS COUNTERPART                    | TERRAFORM MODULE → CONSUMER                                                              | FAITHFUL                                                                        | NOT FAITHFUL — AND WHAT THAT MEANS                                                                                                                                                                                                        |
|---------------------------------------|------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| k3d node container                    | EKS managed node group             | eks → outputs cluster_endpoint, oidc_provider_arn; consumed by iam and the addons module | Kubernetes API behaviour, scheduling, probes, quotas, RBAC                      | Single node, so PodDisruptionBudgets, anti-affinity and node drains are configured but never exercised. Pod IPs come from a Docker bridge, not from VPC subnets, so <b>VPC CNI IP exhaustion cannot occur here</b> .                      |
| ingress-nginx hostPort                | ALB + AWS Load Balancer Controller | network → public_subnet_ids; consumed by the controller through subnet discovery tags    | Ingress objects, path routing, ingress class, nginx timeouts                    | No target groups, no cross-zone balancing, no deregistration delay. A missing kubernetes.io/role/elb tag silently produces no load balancer — I can explain it, I have not hit it.                                                        |
| cert-manager self-signed CA           | ACM certificate on the ALB         | acm → certificate_arn; consumed by the Ingress annotation                                | Certificate lifecycle, secret mounting, renewal mechanics                       | TLS terminates <b>inside</b> the cluster here and <b>at the load balancer</b> in the target. Cipher policy, SNI and WAF association are untested.                                                                                         |
| Postgres StatefulSet + local-path PVC | RDS PostgreSQL, Multi-AZ           | rds → endpoint, port, secret_arn; consumed by the app Helm values and IRSA policy        | Schema, Flyway migrations, connection pooling, credential isolation per service | No failover, no automated backup, no parameter groups. The volume is node-local, so the "volume follows the pod" property of EBS does not exist. Multi-AZ failover is a DNS flip that pools must survive — explainable, not reproducible. |
| Plaintext Secret in Git               | Secrets Manager read via IRSA      | iam → irsa_role_arn; consumed by the ServiceAccount annotation                           | Nothing yet — this is a Phase 1 placeholder                                     | The single largest gap in the repository today. Phase 5 replaces it with External Secrets + SOPS. Until then this is stated as a known limitation, not presented as a design.                                                             |
| Local image import into k3d           | ECR with immutable tags            | ecr → repository_urls; consumed by CI and the deployment image field                     | SHA-tagged images, no :latest, scan-on-build via Trivy                          | No registry authentication, no pull-through cache, no VPC endpoint for private pulls, no lifecycle policy for untagged images.                                                                                                            |

# | Phase 1 — what exists, what is next

## BUILT

- **catalog-service** — Java 21, Spring Boot 3.3, Flyway-owned schema, Hibernate in validate mode, DTOs separate from entities, join-fetch queries to avoid N+1, actuator liveness/readiness probe groups, Prometheus registry already wired.
- **Seed data** — twelve species with real care parameters (temperature, pH, dGH, adult size, minimum group, temperament, diet, plant safety) plus plants, hardscape, equipment and food. A CHECK constraint makes a livestock row without a care profile impossible.
- **storefront** — TypeScript Fastify BFF, server-rendered HTML, 2 s bounded upstream calls, 60 s nav cache, liveness that does *not* depend on the catalog.
- **Containers** — multi-stage, distroless, non-root, read-only root filesystem, dropped capabilities, MaxRAMPercentage so the JVM sizes its heap from the cgroup limit.
- **Cluster** — k3d with Traefik, servicelb and metrics-server disabled; ingress-nginx; cert-manager; ResourceQuota and LimitRange on the namespace.
- **Bootstrap** — one idempotent script with a memory preflight that refuses to run rather than letting the OOM killer make the decision.

## NEXT PHASES

|         |                                                                                                                                                |
|---------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase 2 | inventory-service (Go) with tank-scoped, TTL-bounded, idempotent reservations. Ends with a hold that expires and releases stock on its own.    |
| Phase 3 | order-service and payment-service: the checkout saga with every compensation path demonstrable on demand.                                      |
| Phase 4 | GitOps repo, Argo CD app-of-apps, CI pipelines. Ends when a merged PR deploys with no manual kubectl.                                          |
| Phase 5 | Observability: OTel across all services, RED plus business metrics, burn-rate alerts on the checkout SLO, runbooks. Secrets handling replaced. |
| Phase 6 | aquatics-advisor, notification-service, staff-portal on WildFly — the slow-start WAR exercise.                                                 |
| Phase 7 | Terraform modules and the full local-to-cloud document; NetworkPolicies, PDBs, HPA on a custom RPS metric.                                     |

## KNOWN LIMITATIONS, STATED PLAINLY

- Secrets are plaintext in Git at Phase 1.
- Single-node cluster: PDBs, anti-affinity and drains are untested.
- No environment has run concurrently with another.
- The AWS layer has never been applied; module wiring is proven, AWS behaviour is not.
- Every performance figure in this document set is a placeholder until measured.